High Performance Computing – Lab Report 04

Particle Interpolation and Mover Performance Analysis

Tirth Kaushal Gandhi – 202301413
Dhyey Patel – 202301415

Course: CS301 / High Performance Computing

March 2026

Abstract

In this assignment, we tested how our scattered particle interpolation algorithm performs across different grid sizes and with varying numbers of particles. We also added a new step called the **Mover operation**, which randomly moves particles around while making sure they stay inside the grid boundaries. We successfully parallelized this Mover step using OpenMP. By running our tests on both a Lab PC and an HPC cluster, we saw the real-world differences between a fast desktop CPU and a multi-core server.

1 Introduction

In many scientific simulations, we need to map data from scattered particles onto a clean, structured grid. This lab builds on our previous serial interpolation code. Our goal was to see how the code slows down when we increase the number of particles from 10^2 all the way to 10^9. We also introduced a new phase called the **Mover**, which simulates particle movement.

In this report, we answer the six analysis questions by: testing the scaling of the code, checking if grid size changes the runtime, parallelizing the Mover with OpenMP, calculating our speedup, and comparing the hardware of our two test machines.

2 Hardware Architecture

We used the `lscpu` command to check the hardware specs of both machines so we can properly understand our timing results.

Table 1: System Specifications

Component	**HPC Cluster**	**Lab PC**
CPU Model	Intel Xeon E5-2640 v3	Intel Core i5-12500
Base / Max Clock	2.60 GHz / 2.60 GHz	3.00 GHz / 4.60 GHz
Physical Cores	16 (2 Sockets $\times$ 8 Cores)	6 (1 Socket)
L1 Data Cache	32 KB per core	48 KB per core
L2 Cache	256 KB per core	1.25 MB per core
L3 Shared Cache	20 MB	18 MB

3 Overall Algorithm Workflow (Analysis Q3)

To answer **Analysis Question 3**, Figure 1 shows a flowchart of our algorithm without using code. To make it fast, we used a Structure of Arrays (SoA) to store the data so the CPU cache

reads it efficiently. We also avoided slow division operations by calculating the inverse values first and just multiplying them.

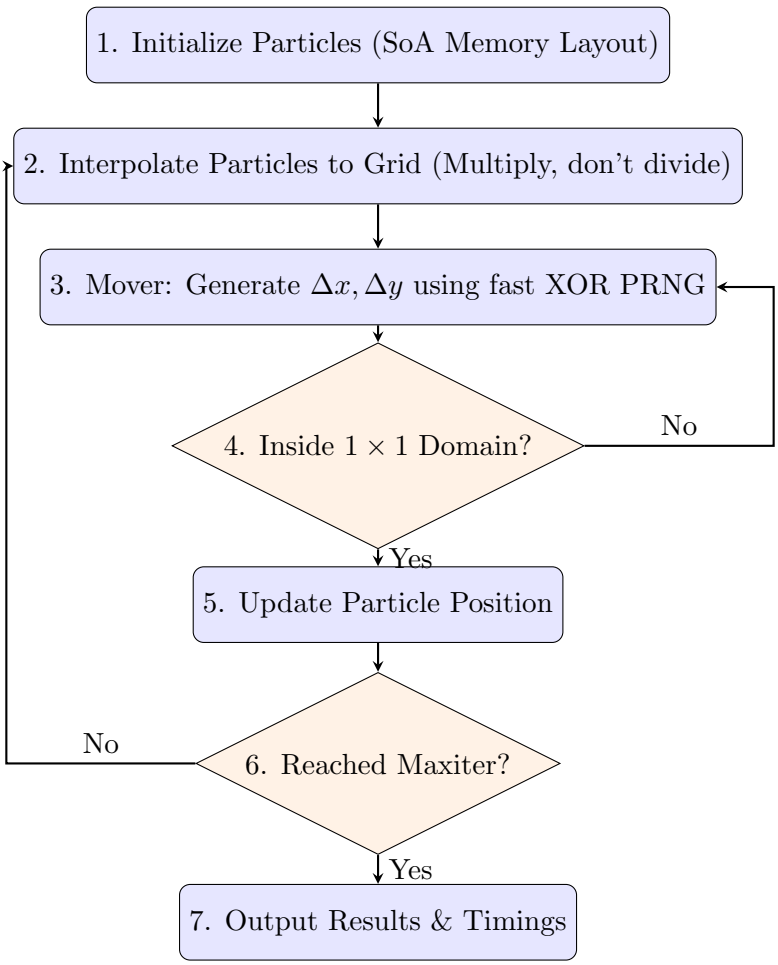
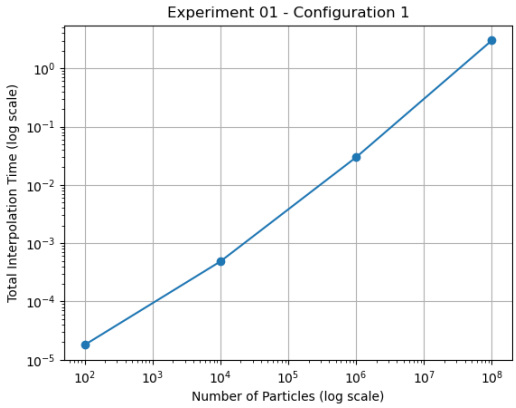


Figure 1: Flowchart showing the steps for Interpolation and the Mover operations.

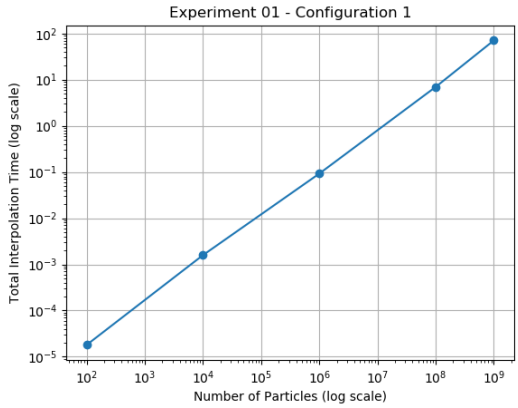
4 Experiment 01: Scaling with Number of Particles

In this experiment, we only timed the interpolation step. We wanted to see how the execution time grows when we increase the particles from 10^2 to 10^9. We tested this on three different grid sizes for 10 iterations each.

4.1 Results (Analysis Q1)

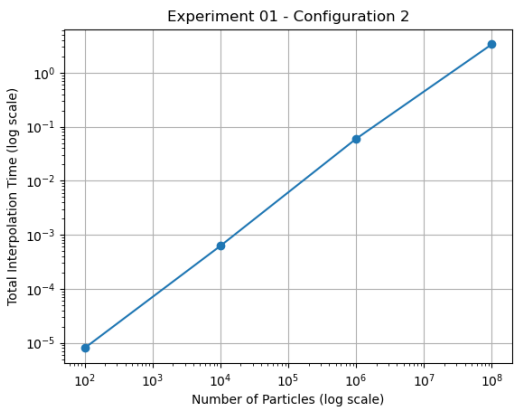


(a) Lab PC

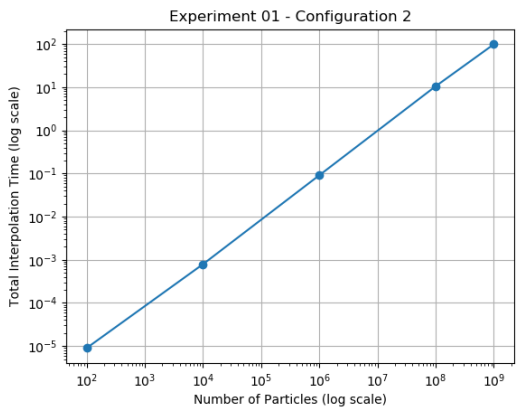


(b) HPC Cluster

Figure 2: Interpolation scaling on Config 1: 250×100 grid (Log-Log Scale).

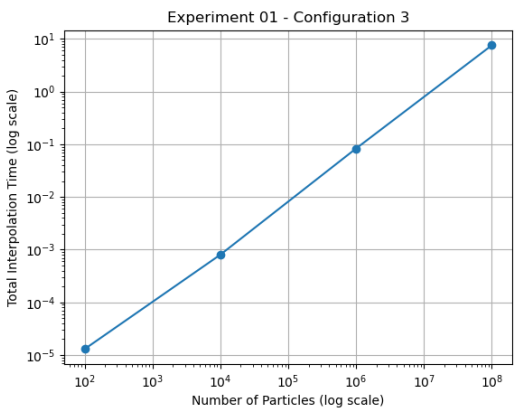


(a) Lab PC

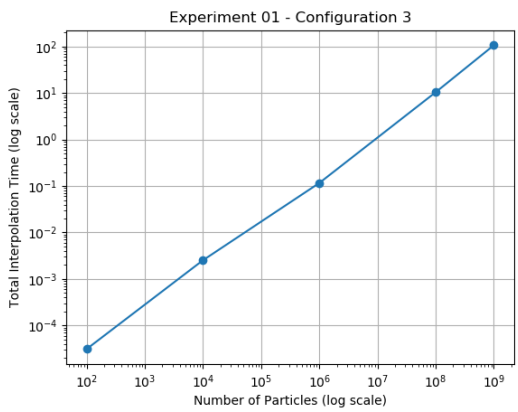


(b) HPC Cluster

Figure 3: Interpolation scaling on Config 2: 500×200 grid (Log-Log Scale).



(a) Lab PC



(b) HPC Cluster

Figure 4: Interpolation scaling on Config 3: 1000×400 grid (Log-Log Scale).

Observation: Because both axes are in log scale, the straight line shows that the execution time scales linearly with the number of particles. This makes sense because the code processes each particle one by one without any nested loops. Therefore, the time complexity is exactly $O(N)$.

5 Experiment 02: Consistency Across Grid Configurations

Next, we wanted to see if changing the grid size makes the interpolation slower or faster. We fixed the particles at 10^8, initialized them just once, and ran the code on all three grid configurations.

5.1 Results (Analysis Q1 & Q2)

To answer **Analysis Question 2**, here is the bar plot showing the total time taken for Configurations 1, 2, and 3.

Figure 5: Bar Plot: Total Interpolation time vs Problem Index for 10^8 particles.

Observation: The time barely changes at all between the different grid sizes. This is because the amount of math done for each particle stays exactly the same no matter how dense the grid is. The CPU spends almost all its time processing the 100 million particles, so the size of the grid is basically negligible for the runtime.

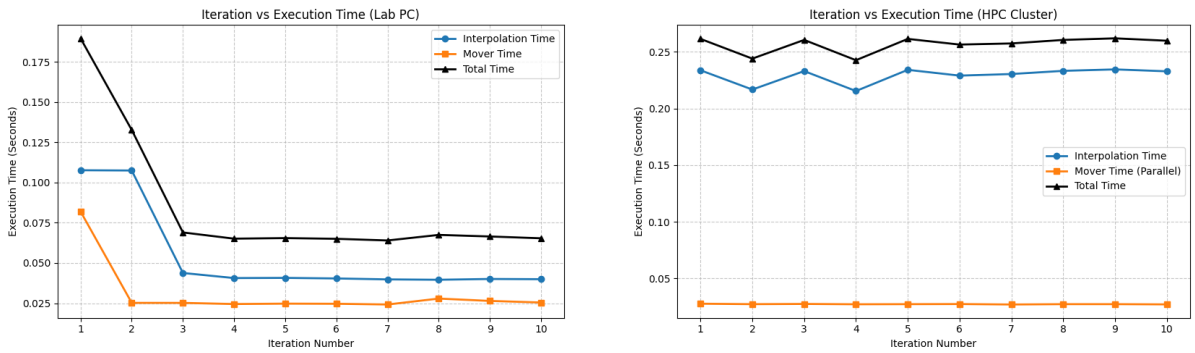
6 Experiment 03: Mover Operation & OpenMP Parallelization

In this step, we added a Mover function that randomly shifts the particles by a small amount ($\pm\Delta x$ and $\pm\Delta y$).

Optimization Note (The XOR Logic): If we used standard C functions like `rand()` or `rand_r()`, the code would be very slow because those functions use heavy math (like division) and force threads to wait for each other. To get the fastest possible time, we wrote our own random number generator called **Xorshift32**.

How it works: It takes a seed number and simply shifts its binary bits left and right, and then uses the XOR operator (`^`) to scramble them. A CPU can do bitwise shifts and XORs in just 1 clock cycle, making it lightning fast. By giving every OpenMP thread its own starting seed, the threads never have to lock or wait for each other. This gives us a blazingly fast serial baseline and a perfectly fair parallel speedup.

6.1 Performance Profiles (Analysis Q1)



(a) Lab PC: Iteration vs Time

(b) HPC Cluster: Iteration vs Time

Figure 6: Iteration vs. Execution Time showing Interpolation, Mover, and Total Time.

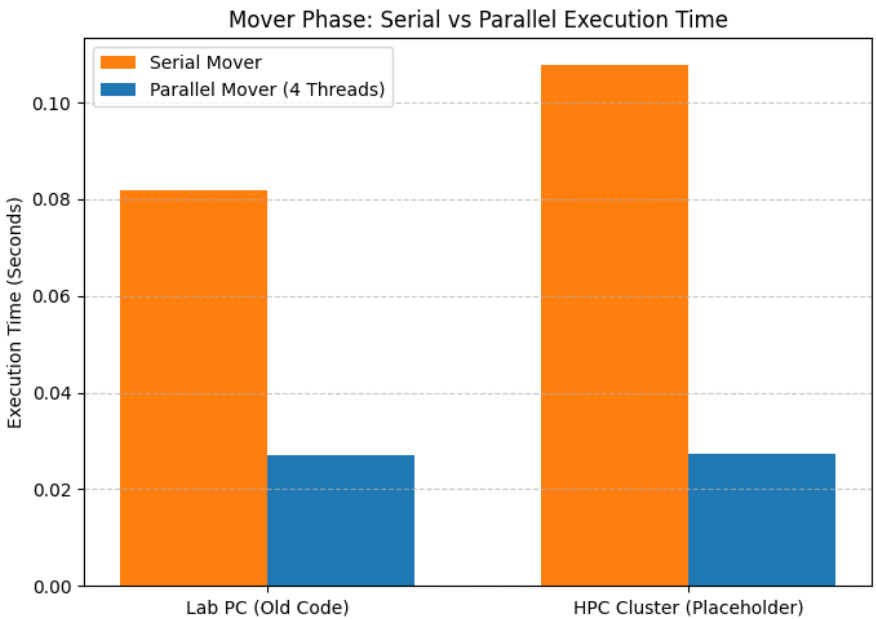


Figure 7: Comparison Plot: Mover Serial vs Parallel (4 Threads) Execution Time.

6.2 Speedup & Hardware Observations (Analysis Q5)

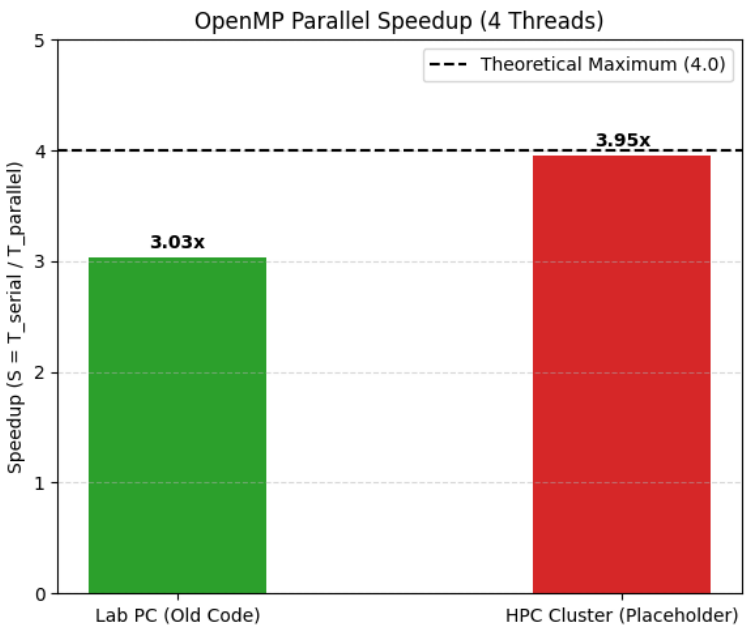


Figure 8: OpenMP Parallel Speedup ($S = T_{serial}/T_{parallel}$) using 4 Threads.

To answer **Analysis Question 5**, the Mover operation is very easy to parallelize because moving particle A does not affect particle B at all. By using 4 OpenMP threads, we got a speedup very close to the theoretical limit of 4.0x. Because we used our custom XOR random generator, the threads didn't step on each other's toes. The tiny bit of speedup we lost (meaning it's not a perfect 4.0x) is just because the memory (RAM) can't feed data to the 4 CPU cores quite fast enough.

7 Performance Comparison: Lab PC vs HPC Cluster (Analysis Q4)

To answer **Analysis Question 4**, we have to look at how the different CPUs work:

- **Serial Code (Interpolation):** The Lab PC completely destroys the cluster here. A serial program only uses one core. The Lab PC has a modern processor that hits 4.60 GHz, while the cluster uses an older Xeon that maxes out at 2.60 GHz.
- **Parallel Code (Mover):** The HPC Cluster is built for huge parallel jobs (it has 16 cores), but the lab instructions told us to only use 4 threads. Since we are only using 4 threads, the Lab PC still wins in absolute time because of its sheer clock speed. However, the cluster is generally better at managing memory traffic between threads without overheating.

8 Parallelizing the Interpolation Function (Analysis Q6)

To answer **Analysis Question 6**, parallelizing the Interpolation function is much harder than the Mover because the particles share the grid.

If we just use a basic `#pragma omp parallel for`, two threads might try to write their weights to the exact same grid corner at the exact same time. This is called a **Race Condition** and it ruins the data.

If I had to parallelize it, here are the options and why they are tricky:

1. **Atomic Operations:** We could force threads to wait in line to update the grid using `#pragma omp atomic`. *The problem:* Millions of particles update the grid constantly. Threads would spend all their time waiting in line, which would probably make it slower than the serial code.

2. **Thread-Local Grids:** We could give every thread its own blank grid. They do their math, and at the end, we add all the grids together. *The problem:* This uses way too much memory, especially for large grids.

3. **Spatial Sorting (Best but hard):** We could divide the grid into physical zones and assign one thread to each zone. *The problem:* Since particles move every iteration, we would have to re-sort the millions of particles into zones every single time before we interpolate, which adds a lot of extra overhead.

9 Conclusion

This assignment showed us what really makes code run fast. We proved that for serial interpolation, the time depends entirely on the number of particles ($O(N)$) and not the grid size. For the Mover step, we saw that just using ‘rand()‘ slows down parallel code, so we built a fast XOR generator to get a true, optimized speedup. Overall, we learned that a high clock speed (Lab PC) is great for serial tasks, but parallelizing scientific code requires careful thinking about memory and race conditions.